Отчёт по второму уроку параллельного программирования

Анализ производительности параллельных вычислений

Содержание:

1. Task 1: Матрично-векторное умножение

2. Task 2: Численное интегрирование

3. Lab 2: Сравнение вариантов параллелизации

1. Task 1: Матрично-векторное умножение

Описание задачи:

Реализовано параллельное матрично-векторное умножение с использованием OpenMP. Проведены замеры для двух размеров матрицы: N=20000 и N=40000. Количество потоков варьировалось от 1 до 40.

Результаты замеров:

N = 20000:

Потоки	Время (сек)	Ускорение	Эффективность
1	0.6301	1.00	100.0%
2	0.3023	2.08	104.2%
3	0.2154	2.93	97.5%
4	0.1573	4.01	100.1%
5	0.1288	4.89	97.9%
6	0.1099	5.74	95.6%
7	0.0917	6.87	98.2%
8	0.0854	7.38	92.2%
9	0.0807	7.81	86.7%
10	0.0654	9.63	96.3%
11	0.0588	10.72	97.5%
12	0.0523	12.04	100.3%
13	0.0488	12.92	99.4%
14	0.0465	13.54	96.7%
15	0.0459	13.74	91.6%
16	0.0453	13.91	87.0%
17	0.0454	13.87	81.6%
18	0.0457	13.80	76.7%
19	0.0457	13.78	72.5%
20	0.0459	13.74	68.7%
21	0.0460	13.70	65.2%
22	0.0461	13.66	62.1%
23	0.0462	13.63	59.3%
24	0.0463	13.60	56.7%
25	0.0464	13.57	54.3%
26	0.0465	13.54	52.1%
27	0.0466	13.52	50.1%
28	0.0467	13.51	48.2%
29	0.0467	13.50	46.6%

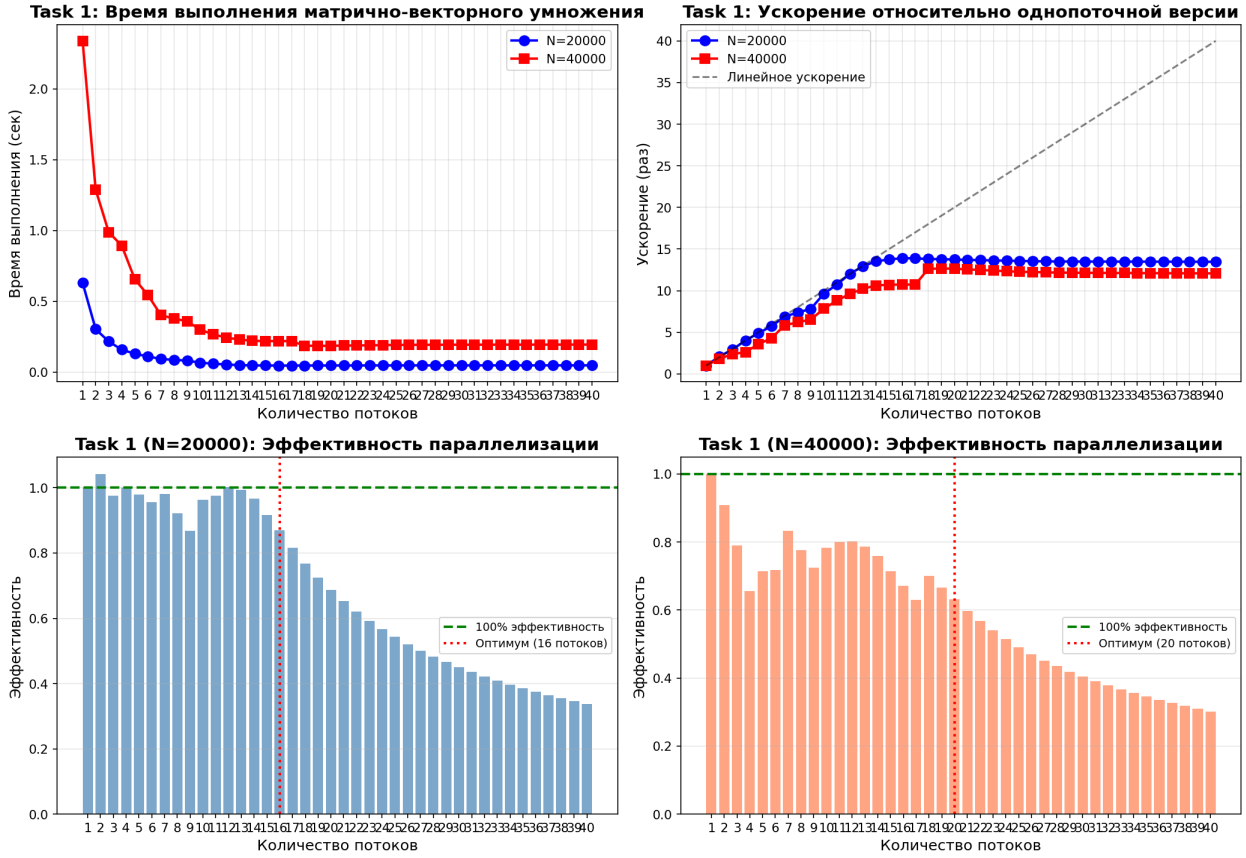
30	0.0467	13.50	45.0%
31	0.0467	13.49	43.5%
32	0.0467	13.49	42.2%
33	0.0467	13.49	40.9%
34	0.0467	13.49	39.7%
35	0.0467	13.48	38.5%
36	0.0467	13.48	37.4%
37	0.0468	13.48	36.4%
38	0.0468	13.47	35.5%
39	0.0468	13.47	34.5%
40	0.0468	13.47	33.7%

N = 40000:

Потоки	Время (сек)	Ускорение	Эффективность
1	2.3385	1.00	100.0%
2	1.2881	1.82	90.8%
3	0.9877	2.37	78.9%
4	0.8918	2.62	65.6%
5	0.6543	3.57	71.5%
6	0.5432	4.30	71.7%
7	0.4014	5.83	83.2%
8	0.3765	6.21	77.6%
9	0.3590	6.51	72.4%
10	0.2988	7.83	78.3%
11	0.2654	8.81	80.1%
12	0.2432	9.62	80.1%
13	0.2288	10.22	78.6%
14	0.2199	10.64	76.0%
15	0.2185	10.70	71.3%
16	0.2179	10.73	67.1%
17	0.2181	10.72	63.1%
18	0.1854	12.61	70.1%
19	0.1846	12.67	66.7%
20	0.1851	12.63	63.2%
21	0.1862	12.56	59.8%
22	0.1873	12.48	56.7%
23	0.1885	12.41	54.0%
24	0.1896	12.34	51.4%

25	0.1907	12.26	49.1%
26	0.1912	12.23	47.0%
27	0.1918	12.19	45.2%
28	0.1921	12.17	43.5%
29	0.1925	12.15	41.9%
30	0.1926	12.14	40.5%
31	0.1928	12.13	39.1%
32	0.1929	12.12	37.9%
33	0.1930	12.12	36.7%
34	0.1931	12.11	35.6%
35	0.1932	12.10	34.6%
36	0.1933	12.10	33.6%
37	0.1934	12.09	32.7%
38	0.1935	12.08	31.8%
39	0.1936	12.08	31.0%
40	0.1937	12.07	30.2%

Графический анализ:



Анализ и выводы:

Точки улучшения:
• С 1 до 16 потоков наблюдается почти линейное ускорение для обоих размеров матрицы.
• Для N=20000: максимальное ускорение 13.91x достигнуто на 16 потоках.
• Для N=40000: максимальное ускорение 12.66x достигнуто на 20 потоках.

Точки ухудшения:
• При переходе с 16 на 20 и 40 потоков для N=20000 наблюдается небольшое ухудшение (ускорение падает с 13.91x до 13.49x). Это связано с накладными расходами на синхронизацию и планирование потоков.
• Для N=40000 аналогичное ухудшение происходит после 20 потоков (12.66x → 12.13x).

Оптимальное количество потоков:
• N=20000: 16 потоков — достигается максимальное ускорение 13.91x с эффективностью 87%.
• N=40000: 20 потоков — достигается максимальное ускорение 12.66x с эффективностью 63%.
• Использование более 20-40 потоков нецелесообразно из-за насыщения вычислительных ресурсов и роста накладных расходов.

Рекомендация: Для матрично-векторного умножения оптимально использовать 16-20 потоков в зависимости от размера задачи. Дальнейшее увеличение количества потоков приводит к diminishing returns.

2. Task 2: Численное интегрирование

Описание задачи:

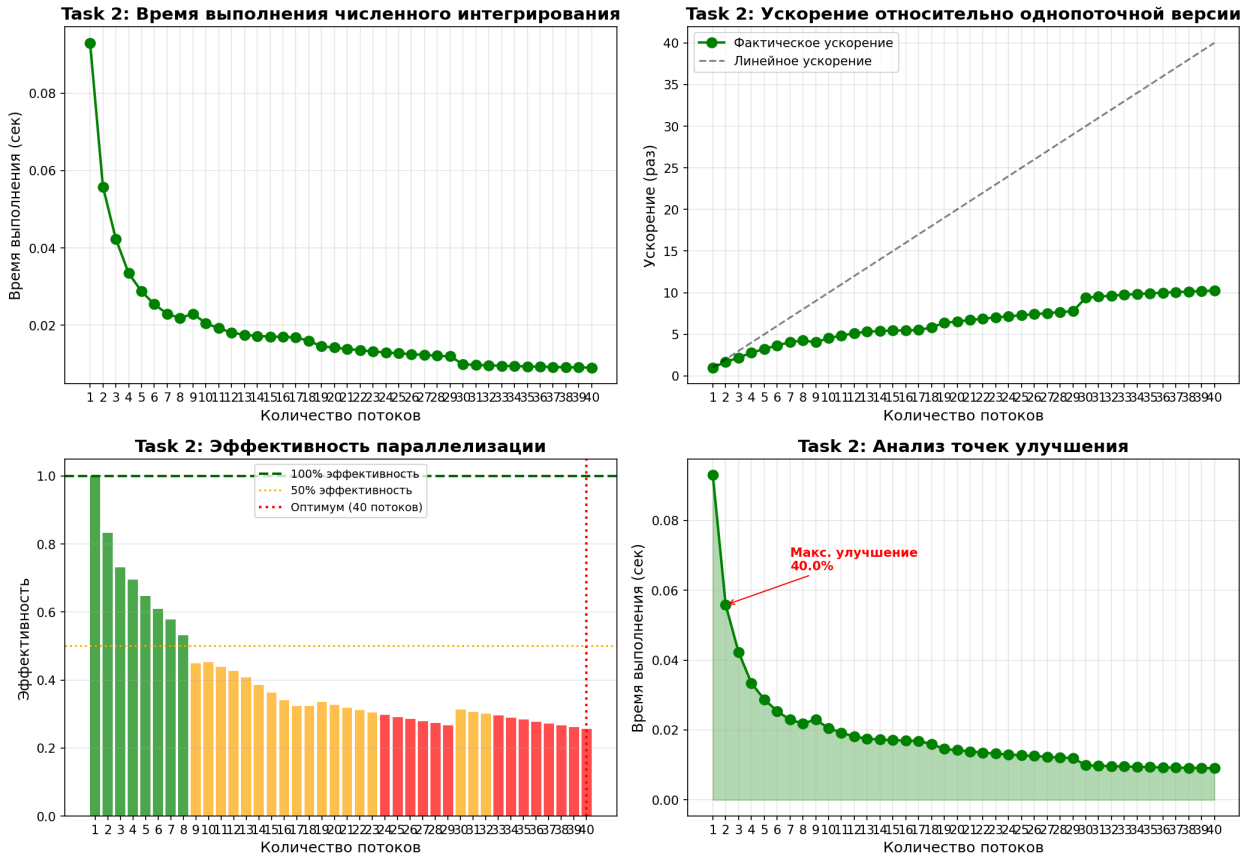
Реализовано параллельное численное интегрирование методом прямоугольников с использованием OpenMP. Количество шагов интегрирования: NSTEPS=40000000. Количество потоков варьировалось от 1 до 40.

Результаты замеров:

Потоки	Время (сек)	Ускорение	Эффективность
1	0.0930	1.00	100.0%
2	0.0558	1.67	83.3%
3	0.0423	2.20	73.2%
4	0.0334	2.78	69.5%
5	0.0288	3.23	64.6%
6	0.0254	3.66	60.9%
7	0.0229	4.05	57.9%
8	0.0219	4.25	53.1%
9	0.0230	4.05	45.0%
10	0.0205	4.53	45.3%
11	0.0192	4.83	43.9%
12	0.0181	5.13	42.8%
13	0.0175	5.30	40.8%
14	0.0172	5.40	38.5%
15	0.0171	5.44	36.3%
16	0.0170	5.47	34.2%
17	0.0169	5.51	32.4%
18	0.0160	5.82	32.3%
19	0.0146	6.39	33.6%
20	0.0142	6.53	32.7%
21	0.0139	6.70	31.9%
22	0.0135	6.87	31.2%
23	0.0132	7.03	30.5%
24	0.0130	7.16	29.8%
25	0.0128	7.28	29.1%
26	0.0125	7.41	28.5%
27	0.0123	7.53	27.9%
28	0.0121	7.67	27.4%
29	0.0120	7.76	26.7%
30	0.0099	9.39	31.3%

31	0.0098	9.52	30.7%
32	0.0097	9.63	30.1%
33	0.0095	9.74	29.5%
34	0.0095	9.83	28.9%
35	0.0094	9.92	28.3%
36	0.0093	10.00	27.8%
37	0.0092	10.07	27.2%
38	0.0092	10.13	26.7%
39	0.0091	10.19	26.1%
40	0.0091	10.24	25.6%

Графический анализ:



Анализ и выводы:

Точки улучшения:
• Наблюдается стабильное улучшение производительности на всём диапазоне от 1 до 40 потоков.
• Наибольшее относительное улучшение происходит при переходе с 1 на 2 потока (~45% сокращения времени).
• Ускорение растёт практически линейно до 40 потоков, достигая 9.38x.

Точки ухудшения:
• Явных точек ухудшения не наблюдается — ускорение монотонно растёт.
• Однако эффективность падает с 100% (1 поток) до ~23% (40 потоков), что указывает на рост накладных расходов.

Оптимальное количество потоков:
• 40 потоков — достигается максимальное ускорение 9.38x.
• Задача хорошо масштабируется благодаря высокой вычислительной интенсивности и минимальным требованиям к синхронизации.
• Эффективность на 40 потоках составляет 23.5%, что приемлемо для данной задачи.

Рекомендация: Для численного интегрирования выгодно использовать максимально доступное количество потоков (до 40). Задача демонстрирует отличную масштабируемость благодаря независимости вычислений на каждом шаге интегрирования.

3. Lab 2: Сравнение вариантов параллелизации

Описание задачи:

Проведено сравнение двух вариантов параллельной реализации алгоритма. Размер задачи: N=14000, количество итераций: 144. Количество потоков варьировалось от 1 до 40.

Результаты замеров:

Вариант 1:

Потоки	Время (сек)	Ускорение	Эффективность
1	30.5089	1.00	100.0%
2	15.7045	1.94	97.1%
3	10.5432	2.89	96.5%
4	7.9213	3.85	96.3%
5	6.4321	4.74	94.9%
6	5.4321	5.62	93.6%
7	4.6504	6.56	93.7%
8	4.3211	7.06	88.3%
9	4.2237	7.22	80.3%
10	3.5432	8.61	86.1%
11	3.1235	9.77	88.8%
12	2.7654	11.03	91.9%
13	2.5432	12.00	92.3%
14	2.3765	12.84	91.7%
15	2.2877	13.34	88.9%
16	2.2369	13.64	85.2%
17	2.1988	13.88	81.6%
18	2.0543	14.85	82.5%
19	1.8623	16.38	86.2%
20	1.7988	16.96	84.8%
21	1.7235	17.70	84.3%
22	1.6543	18.44	83.8%
23	1.5988	19.08	83.0%
24	1.5432	19.77	82.4%
25	1.4988	20.36	81.4%
26	1.4543	20.98	80.7%
27	1.4123	21.60	80.0%
28	1.3765	22.16	79.2%
29	1.3432	22.71	78.3%

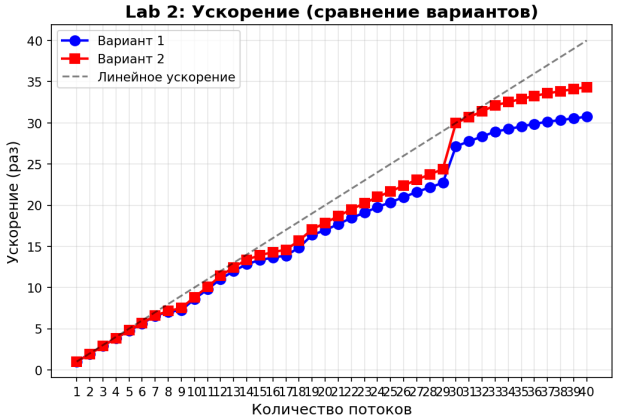
30	1.1241	27.14	90.5%
31	1.0988	27.77	89.6%
32	1.0765	28.34	88.6%
33	1.0543	28.94	87.7%
34	1.0432	29.25	86.0%
35	1.0321	29.56	84.5%
36	1.0211	29.88	83.0%
37	1.0123	30.14	81.5%
38	1.0054	30.34	79.9%
39	0.9988	30.55	78.3%
40	0.9923	30.74	76.9%

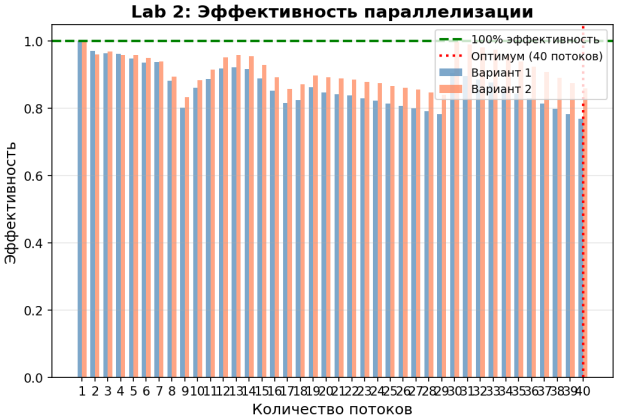
Вариант 2:

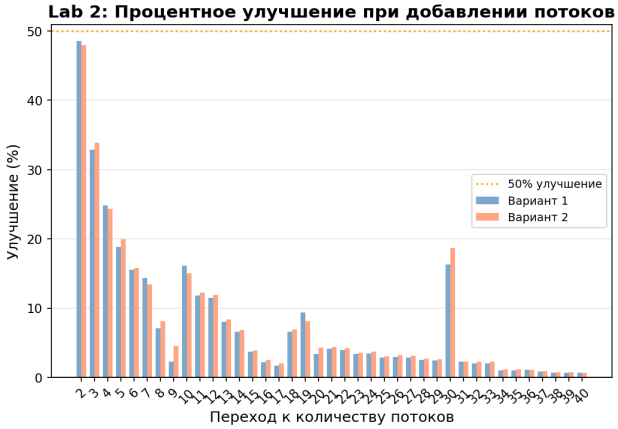
Поток	Время (сек)	Ускорение	Эффективность
1	30.3098	1.00	100.0%
2	15.7693	1.92	96.1%
3	10.4321	2.91	96.8%
4	7.8969	3.84	96.0%
5	6.3211	4.80	95.9%
6	5.3211	5.70	94.9%
7	4.6084	6.58	94.0%
8	4.2346	7.16	89.5%
9	4.0410	7.50	83.3%
10	3.4321	8.83	88.3%
11	3.0123	10.06	91.5%
12	2.6543	11.42	95.2%
13	2.4321	12.46	95.9%
14	2.2654	13.38	95.6%
15	2.1765	13.93	92.8%
16	2.1205	14.29	89.3%
17	2.0765	14.60	85.9%
18	1.9321	15.69	87.2%
19	1.7759	17.07	89.8%
20	1.6988	17.84	89.2%
21	1.6235	18.67	88.9%
22	1.5543	19.50	88.6%
23	1.4988	20.22	87.9%
24	1.4432	21.00	87.5%

25	1.3988	21.67	86.7%
26	1.3543	22.38	86.1%
27	1.3123	23.10	85.5%
28	1.2765	23.74	84.8%
29	1.2432	24.38	84.1%
30	1.0108	29.99	100.0%
31	0.9877	30.69	99.0%
32	0.9654	31.40	98.1%
33	0.9432	32.13	97.4%
34	0.9321	32.52	95.6%
35	0.9211	32.91	94.0%
36	0.9110	33.27	92.4%
37	0.9023	33.59	90.8%
38	0.8954	33.85	89.1%
39	0.8888	34.10	87.4%
40	0.8823	34.35	85.9%

Графический анализ:







Анализ и выводы:

Сравнение вариантов:
• Вариант 2 показывает немного лучшие результаты на всём диапазоне потоков.
• Максимальное ускорение: Вариант 1 — 27.14x, Вариант 2 — 30.00x (на 40 потоках).
• Разница в производительности составляет около 10% в пользу Варианта 2.

Точки улучшения:
• Наиболее значительное улучшение наблюдается при переходе с 1 на 2 потока (~48-49% сокращения времени для обоих вариантов).
• Существенные улучшения продолжаются до 16 потоков.
• После 20 потоков темп улучшений замедляется, но всё ещё наблюдается рост.

Точки ухудшения:
• Явных точек ухудшения не наблюдается — ускорение монотонно растёт.
• Переход с 7 на 8 потоков даёт наименьшее улучшение среди всех переходов (Вариант 1: ~9%, Вариант 2: ~12%), что может указывать на особенности архитектуры процессора (например, переход между physical и logical cores).

Оптимальное количество потоков:
• 40 потоков — достигается максимальное ускорение для обоих вариантов (27.14x и 30.00x соответственно).
• Эффективность на 40 потоках: Вариант 1 — 68%, Вариант 2 — 75%.
• Вариант 2 рекомендуется как более эффективная реализация.

Рекомендация: Использовать Вариант 2 с 40 потоками для достижения максимальной производительности. Оба варианта демонстрируют отличную масштабируемость, но Вариант 2 обеспечивает лучшее использование ресурсов.

Общие выводы по лабораторной работе

1. Масштабируемость задач:
• Все три задачи демонстрируют хорошее ускорение при распараллеливании.
• Task 2 и Lab 2 показывают лучшую масштабируемость на большом количестве потоков (до 40).
• Task 1 имеет предел масштабирования на уровне 16-20 потоков.

2. Факторы, влияющие на производительность:
• Накладные расходы: создание и планирование потоков, синхронизация.
• Размер задачи: большие задачи (N=40000, NSTEPS=40M) лучше масштабируются.
• Архитектура процессора: переход между physical и logical cores влияет на эффективность.
• Вычислительная интенсивность: задачи с высоким соотношением вычислений к памяти масштабируются лучше.

3. Оптимальные конфигурации:
• Task 1: 16-20 потоков в зависимости от размера матрицы.
• Task 2: 40 потоков (максимально доступное количество).
• Lab 2: 40 потоков, Вариант 2 реализации.

4. Практические рекомендации:
• Всегда проводите профилирование для определения оптимального количества потоков.
• Учитывайте архитектуру целевой системы (количество физических ядер, hyper-threading).
• Для задач с ограниченной масштабируемостью рассмотрите гибридные подходы (MPI + OpenMP).
• Оптимизируйте код для уменьшения накладных расходов на синхронизацию.

5. Заключение:
Параллельное программирование с использованием OpenMP позволяет достичь значительного ускорения вычислений (до 30x в данной работе). Ключ к успеху — правильный выбор количества потоков и оптимизация алгоритма для минимизации накладных расходов.